

1. **Θεωρία CSP-M σε 2-3 “σελίδες”** (events, processes, επιλογές, κανάλια, εσωτερική/εξωτερική επιλογή).
2. **Πίνακας συμβόλων CSP-M + τι σημαίνει το καθένα.**
3. **Βήμα-βήμα σύνθεση για πρόγραμμα που τρέχει στο FDR4.**
4. **Πλήρες παράδειγμα vending machine** (CSP-M που τρέχει στο FDR4, με εξήγηση γραμμή-γραμμή).
5. **Πλήρες παράδειγμα bank account** στο στυλ του Exercise 2 (με delay σε large deposit + fraud).

1. Συνοπτική θεωρία CSP / CSP-M (με βάση Lectures 2–5)

1.1 Βασικές έννοιες: events, alphabets, processes

- **Event:** ένα “στιγμιαίο” συμβάν στο σύστημα (π.χ. coin, choc, deposit, withdraw).
Δεν έχει διάρκεια, είναι ατομικό.

lecture 2

- **Alphabet (A):** το σύνολο των events που *μπορεί* να εκτελέσει ένας process.
Πάντα ισχύει $A \subseteq \Sigma$, όπου Σ = όλα τα events του κόσμου.

lecture 2

- **Process:** περιγραφή συμπεριφοράς: “πότε” επιτρέπονται ποια events. Δεν κρατάει μόνο τι γίνεται, αλλά και τι *απαγορεύεται* να γίνει.

lecture 2

Παράδειγμα (θεωρητικό):

```
Student =  
  getup ->  
  lecture ->  
  laboratory ->  
  eat ->  
  sleep ->  
  Student
```

Ο Student επαναλαμβάνει αυτόν τον κύκλο για πάντα.

lecture 2

1.2 Βασικοί τελεστές CSP

STOP

- `STOP` = process που **δεν κάνει τίποτα ποτέ**. Δεν επικοινωνεί, δεν τερματίζει.

lecture 2

Prefix `a -> P`

- `a -> P` = “περιμένω το event `a`, μετά συμπεριφέρομαι σαν τον `P`”.

lecture 2

Παράδειγμα:

```
Day =  
  getup -> breakfast -> seminar -> STOP
```

Ορισμός process

- Γράφουμε `P = a -> Q [] b -> R` για να “βαφτίσουμε” τον process `P` ως συνδυασμό άλλων.

lecture 2

Recursion

Για άπειρη συμπεριφορά:

```
P = up -> down -> P  
CarAlarm = beep -> CarAlarm
```

Η recursion πρέπει να είναι **guarded**: κάθε διαδρομή περνά από event πριν φτάσει ξανά στο ίδιο όνομα.

lecture 2

External choice `[]`

Δίνει στην **environment** (δηλαδή στον “έξω κόσμο”) την επιλογή ποιο event θα συμβεί πρώτο.

```
UpAndDown =  
  up -> down -> STOP  
  []  
  down -> up -> STOP
```

Ο κόσμος διαλέγει: είτε up πρώτα, είτε down πρώτα.

Internal choice $| \sim |$

Είναι μη-ντετερμινιστική εσωτερική επιλογή: το σύστημα αποφασίζει “από μέσα”, χωρίς να μπορεί να το ελέγξει η environment.

lecture 3

```
P = a -> STOP |~| b -> STOP
```

Η environment βλέπει ή μια διαδικασία που θα κάνει a, ή μια διαδικασία που θα κάνει b, αλλά δεν ελέγχει ποια.

1.3 Κανάλια – input/output (CSP-M)

Στο CSP-M (που διαβάζει το FDR4) έχουμε **κανάλια**:

```
channel c : T          -- c επικοινωνεί τιμές τύπου T
```

- **Είσοδος:** $c?x:T \rightarrow P(x)$ (διαβάζω από το κανάλι c μια τιμή τύπου T στη μεταβλητή x).

lecture 3

- **Έξοδος:** $c!v \rightarrow P$ (στέλνω την τιμή v μέσω του καναλιού c).

lecture 3

Παράδειγμα από lecture 3:

```
Copy = left?x:T -> right!x -> Copy
```

lecture 3

1.4 Conditional choice

Μπορούμε να φιλτράρουμε events με **boolean** συνθήκες:

```
Counter(x,y) =  
  x > 0 & down -> Counter(x-1, y)  
  []  
  x < 7 & up   -> Counter(x+1, y)
```

lecture 3

Αν η συνθήκη είναι false, αυτό το branch είναι σαν STOP.

1.5 Βασικοί “standard” processes

- $\text{RUN}(A) = ?x:A \rightarrow \text{RUN}(A)$: πάντα πρόθυμος να κάνει οποιοδήποτε event από A.

lecture 3

- $\text{CHAOS}(A) = \text{STOP} \mid \sim \mid ?x:A \rightarrow \text{CHAOS}(A)$: μπορεί να δεχτεί ή να απορρίψει οτιδήποτε από A.

lecture 3

1.6 Algebra & Verification (Lecture 5)

- CSP έχει αλγεβρικούς νόμους: π.χ.
 - Idempotence: $P \mid P = P, P \mid \sim \mid P = P$
 - Symmetry: $P \mid Q = Q \mid P$
 - Associativity: $(P \mid Q) \mid R = P \mid (Q \mid R)$
- Αυτοί οι νόμοι δίνουν **σημασιολογία** και επιτρέπουν **refinement & ισοδυναμίες**: αν δύο processes ικανοποιούν τις ίδιες traces / failures / laws, είναι “ίδιοι” από τη σκοπιά της environment.

lecture 5

Το FDR4 χρησιμοποιεί αυτά τα μοντέλα (ιδίως **traces / failures-divergences**) για να κάνει model checking & refinement proofs.

2. CSP-M σύμβολα & τι σημαίνουν (για FDR4)

Βασικά:

Σύμβολο	Σημασία (intuitive)
$\rightarrow$	prefix: πρώτα αυτό το event, μετά αυτός ο process
$\mid$	external choice (επιλογή από περιβάλλον)
$\sim$	$\sim$
STOP	δεν κάνει κανένα event, δεν τερματίζει

Σύμβολο	Σημασία (intuitive)
SKIP	(αν το χρησιμοποιήσεις) σημαίνει “ομαλός τερματισμός”
$?x:T$	input από κανάλι, δέχεται οποιαδήποτε τιμή τύπου T
$!v$	output σε κανάλι, στέλνει τιμή v
$\&$	guard: “αν η συνθήκη είναι true, επιτρέπεται το event”
if ... then ... else ...	κλασικό conditional
$\{a,b,c\}$	σύνολο events
channel coin, choc	ορισμός καναλιών χωρίς τύπο δεδομένων
channel show_balance : Nat	κανάλι που μεταφέρει τιμές τύπου Nat
RUN(A)	process που πάντα μπορεί να κάνει οποιοδήποτε event του A
CHAOS(A)	διαδικασία “πανικού / χάους”, μη προβλέψιμη

3. Πώς γράφω CSP-M πρόγραμμα για FDR4 – Βήμα-βήμα

Βήμα 1: Διαλέγω *events* και *κανάλια*

- Πρώτα σκέφτομαι **τι συμβαίνει** στο σύστημα (κουτιά αυτοματοποιημένου Hoare: “Think of vending machines, bank accounts...”).
- Από αυτά βγάζω events: π.χ. coin, choc, toffee, deposit, withdraw, query.

Στο CSP-M:

```
channel coin, choc, toffee
```

ή με τύπο:

```
channel show_balance : Nat
```

Βήμα 2: Ορίζω αλφάβητο αν το χρειάζομαι

Π.χ.:

```
VMEvents = {coin, choc, toffee}
```

Αυτό βοηθάει σε RUN(A), CHAOS(A) ή σε παραλληλισμό αργότερα.

Βήμα 3: Γράφω τον κεντρικό process με prefix + recursion

Παράδειγμα vending:

```
VENDING =  
  coin -> (  
    choc  -> VENDING  
    []  
    toffee -> VENDING  
  )
```

Προσοχή:

- Κάθε recursion πρέπει να γίνεται *μετά* από event (guarded recursion).

Βήμα 4: (Προαιρετικά) ορίζω περιβάλλον / πελάτη / τράπεζα

```
CUSTOMER =  
  coin -> choc -> CUSTOMER
```

Βήμα 5: Ορίζω τι είναι το σύστημα που θα simulate-άρω

Π.χ.:

```
SYSTEM = VENDING
```

ή:

```
SYSTEM = VENDING [|{|coin, choc, toffee|}|] CUSTOMER
```

Για το online FDR στο cocotec.io/fdr μπορείς απλά να ορίσεις

```
MAIN = SYSTEM
```

και μετά να κάνεις simulate τον MAIN.

Βήμα 6: (Προαιρετικά) προσθέτω asserts για verification

Παράδειγμα:

```
assert SYSTEM :[deadlock free [F]]
```

ή refinement:

```
SPEC = RUN(VMEvents)  
assert SYSTEM [T= SPEC      -- traces refinement
```

Για αρχή, δεν είναι υποχρεωτικό· για **Probe simulation** αρκεί να έχεις processes.

4. Πλήρες παράδειγμα Vending Machine (για FDR4) + εξήγηση

4.1 Κώδικας vending_example.csp

```
-- Simple vending machine in CSP-M (FDR4 compatible)

-- 1. Δηλώνουμε τα events μας ως κανάλια
channel coin, choc, toffee

-- 2. Ορίζουμε το αλφάβητο (προαιρετικό αλλά χρήσιμο)
VMEvents = {coin, choc, toffee}

-- 3. Ο κεντρικός process του μηχανήματος
VENDING =
  coin ->
    ( choc -> VENDING      -- περιμένει ένα νόμισμα
      []                  -- μετά δίνει σοκολάτα
      toffee -> VENDING    -- ή
      )                  -- δίνει καραμέλα

-- 4. Προαιρετικός "πελάτης" που πάντα βάζει νόμισμα και παίρνει
σοκολάτα
CUSTOMER =
  coin -> choc -> CUSTOMER

-- 5. Συνολικό σύστημα (μηχανή + πελάτης σε συγχρονισμό στα κοινά
events)
SYSTEM =
  VENDING [|{|coin, choc, toffee|}] CUSTOMER

-- 6. Τι θα simulate-άρεις
MAIN = SYSTEM

-- (Προαιρετικό) Deadlock freedom check
-- assert MAIN :[deadlock free [F]]
```

4.2 Τι κάνει κάθε κομμάτι

- `channel coin, choc, toffee`
Δηλώνει 3 events που θα εμφανίζονται στα traces.
- `VENDING = coin -> (choc -> VENDING [] toffee -> VENDING)`
Μοντέλο:
 - Πάντα **πρώτα** περιμένω ένα `coin`.
 - Μετά αφήνω το περιβάλλον να επιλέξει: `choc` ή `toffee`.
 - Μετά επιστρέφω στην ίδια κατάσταση → άπειρο loop.
- `CUSTOMER`
Απλός deterministic πελάτης: κάθε φορά `coin` μετά `choc`.
- `SYSTEM = VENDING [|{|coin, choc, toffee|}] CUSTOMER`
Parallel composition με συγχρονισμό στα κοινά events:
 - Ο πελάτης και το μηχάνημα πρέπει να συμφωνήσουν για `coin` & `choc`.
 - Αν ο πελάτης δεν ζητά `toffee`, δεν θα εμφανιστεί ποτέ.

- `MAIN = SYSTEM`
Στο cocotec/FDR4: άνοιξε το αρχείο, διάλεξε `MAIN`, πάτα `Probe` → θα δεις τα βήματα.

5. Πλήρες παράδειγμα Bank Account (Exercise 2)

Στόχος: Μοντέλο που:

1. Επιτρέπει **deposit** (small & large).
2. Επιτρέπει **withdraw** μέχρι το διαθέσιμο balance.
3. Επιτρέπει **query** balance.
4. Για **large deposit**, τα χρήματα πάνε σε “pending” μέχρι `clear_large`.
5. Υπάρχει πιθανότητα **fraud_withdraw** (μη εξουσιοδοτημένη ανάληψη).

6CM504 - Exercise 2

5.1 Ιδέα μοντέλου

- Δύο “μετρητές”:
 - `avail` = ποσό διαθέσιμο για κανονικό `withdraw`.
 - `held` = ποσό σε `hold` (από `large deposits` που δεν έχουν ξεκαθαριστεί).
- Events:
 - `deposit_small`
 - `deposit_large`
 - `withdraw`
 - `fraud_withdraw`
 - `query`
 - `show_balance!n` (βγάζει το διαθέσιμο ποσό)
 - `clear_large` (η τράπεζα “ξεκλειδώνει” `pending` χρήματα)

Χρησιμοποιούμε `Nat` όπως στο counter του lecture 3.

5.2 Κώδικας `bank_example.csp`

```
-- Simple bank account model (FDR4 / CSP-M)

-- 1. Δηλώνουμε events / κανάλια
channel deposit_small, deposit_large, withdraw, query, clear_large,
fraud_withdraw
channel show_balance : Nat

-- 2. Κύριος process με δύο ακέραιες παραμέτρους:
--   avail = διαθέσιμα χρήματα
--   held  = μεγάλα ποσά σε αναμονή (hold)
ACCOUNT(avail, held) =

  -- Μικρή κατάθεση: πάει κατευθείαν στα διαθέσιμα
  deposit_small ->
    ACCOUNT(avail + 1, held)
```



```

[]

-- Μεγάλη κατάθεση: πάει στα held μέχρι να γίνει clear
deposit_large ->
  ACCOUNT(avail, held + 1)

[]

-- Κανονική ανάληψη: μόνο αν έχουμε διαθέσιμο balance
(avail > 0 & withdraw ->
  ACCOUNT(avail - 1, held))

[]

-- Ερώτηση υπολοίπου: δείχνω το avail
query ->
  show_balance!avail ->
  ACCOUNT(avail, held)

[]

-- Clear των μεγάλων deposits (όταν το bank fraud check
ολοκληρωθεί)
(held > 0 & clear_large ->
  ACCOUNT(avail + held, 0))

[]

-- Fraudulent withdrawal: κλέβει από το διαθέσιμο (χωρίς έλεγχο)
(avail > 0 & fraud_withdraw ->
  ACCOUNT(avail - 1, held))

-- 3. Αρχική κατάσταση λογαριασμού: 0 διαθέσιμα, 0 held
BANK = ACCOUNT(0, 0)

-- 4. Αυτόν τον process θα simulate-άρουμε
MAIN = BANK

-- (προαιρετικά) Deadlock freedom, traces, refinement assertions
κ.λπ.
-- assert MAIN :[deadlock free [F]]

```

5.3 Τι σημαίνει κάθε μέρος

- ACCOUNT(avail, held)
Process με “κατάσταση” (avail, held) ενσωματωμένη ως παράμετροι (όπως στο Counter(x, y)).

lecture 3

- deposit_small -> ACCOUNT(avail+1, held)
Καταθέτει “μία μονάδα” (μπορείς να φανταστείς ότι 1 μονάδα = 10€ ή 100€). Δεν μας νοιάζει το νόμισμα, μόνο η συμπεριφορά.
- deposit_large -> ACCOUNT(avail, held+1)
Τα χρήματα δεν είναι άμεσα διαθέσιμα, πάνε στα held.

- `(avail > 0 & withdraw -> ...)`
Guard: αν `avail > 0`, επιτρέπεται το `withdraw`. Αλλιώς αυτό το branch είναι σαν STOP.
 - `query -> show_balance!avail -> ACCOUNT(...)`
Ο πελάτης ζητά `query`, η τράπεζα απαντάει στο `show_balance` το τρέχον διαθέσιμο, μετά ο λογαριασμός συνεχίζει.
 - `(held > 0 & clear_large -> ACCOUNT(avail+held, 0))`
Το bank “περνάει” τα `held` στο διαθέσιμο (`clear large deposit`).
 - `(avail > 0 & fraud_withdraw -> ...)`
Μοντελοποιεί το σενάριο όπου ο λογαριασμός είναι compromised και κάποιος κλέβει χρήματα σαν “κανονική” συναλλαγή (όπως ζητά το exercise).
 - `BANK = ACCOUNT(0, 0)`
Αρχική κατάσταση: 0 balance.
-

5.4 Πώς θα το τρέξεις στο cocotec FDR4

1. Ανοίγεις το <https://cocotec.io/fdr/>
2. Δημιουργείς νέο αρχείο, κάνεις paste τον κώδικα vending ή bank.
3. Πατάς “Check” / “Compile” (ανάλογα UI).
4. Από τη λίστα processes, διαλέγεις **MAIN** και ανοίγεις **Probe**.
5. Κάνεις explore τα events: `deposit_small`, `withdraw`, `query` κ.λπ., να δεις ότι η συμπεριφορά βγάζει νόημα.

ΑΝΑΛΥΤΙΚΟ TEACHING SCRIPT (ό,τι θα έλεγες στην τάξη)

Βασισμένο στα lectures 2–5 και στα examples.

Πλήρως έτοιμο για χρήση σε μάθημα.

ΔΙΑΡΘΡΩΣΗ:

1. Εισαγωγή στη λογική CSP / FDR4
 2. Εξήγηση συμβόλων & σύνταξης CSP-M
 3. Πώς συντάσσουμε πρόγραμμα CSP-M (τεχνική – “συνταγή”)
 4. Teaching Script για VENDING MACHINE (με λεπτομερή ανάλυση)
 5. Teaching Script για BANK ACCOUNT (τύπου Example 2)
 6. Τι να δείξω live στο FDR4 (Probe / Refinement)
-

☆ 1. Εισαγωγή στη λογική CSP

Μοντελοποιούμε συστήματα συμπεριφοράς χρησιμοποιώντας CSP, και πώς τα επαληθεύουμε με FDR4.

Θα δούμε:

- Τι είναι **event**, **process**, **choice**, **channels**
- Πώς γράφουμε CSP-M κώδικα

- Πώς κάνουμε **simulation (Probe)**
- Και θα χτίσουμε δύο μοντέλα:
 1. Ένα **vending machine**
 2. Ένα **bank account** με pending deposits & fraud (Example 2)

Επεξήγηση:

- CSP μοντελοποιεί **συμπεριφορά**, όχι αριθμητικούς υπολογισμούς.
- Κάθε process λέει **τι επιτρέπεται** και **τι απαγορεύεται**.
- Το FDR4 μπορεί να κάνει:
 - καταγραφή αυτής της συμπεριφοράς
 - simulation
 - equivalence checking
 - refinement checking
 - deadlock checking

★ 2. Συμβολισμοί CSP-M

Τα 4 βασικά σύμβολα

Σύμβολο	Τι λες ότι σημαίνει
$a \rightarrow P$	«Περιμένω το event a και μετά συμπεριφέρομαι σαν τον P »
$[]$	«External choice — το περιβάλλον διαλέγει ποιο event θα γίνει»
$\cdot$	$\sim$
STOP	«Καμία συμπεριφορά / deadlock state»

Recursion

«Για να φτιάξω μηχανή που λειτουργεί επ' άπειρον, χρειάζομαι recursion.
ΠΡΟΣΟΧΗ: η recursion πρέπει πάντα να περνάει από event.»

Channels

«Τα channels επιτρέπουν σε processes να επικοινωνήσουν τιμές.
Με $?x:T$ διαβάζω τιμή, με $!v$ στέλνω τιμή.»

Παράδειγμα που εξηγείς:

```
Copy = left?x -> right!x -> Copy
```

«Με κάθε event, ο Copy παίρνει μία τιμή από το left και την γράφει στο right.»

★ 3. Συνταγή για συγγραφή CSP-M

Βήμα 1: Ορίζω events / κανάλια

«Ξεκινάω με το τι συμβαίνει στο σύστημα. Αυτά μετατρέπονται σε events.»

Βήμα 2: Χτίζω τον κεντρικό process

«Ξεκινάω από την πιο απλή συμπεριφορά: prefix → recursion.
Μετά βάζω external choices και guards.»

Βήμα 3: Αν έχει κατάσταση, τη βάζω ως παράμετρο

«Όπως στο counter, βάζω μια παράμετρο που δείχνει state:
ACCOUNT(avail, held)»

Βήμα 4: Δημιουργώ το MAIN

«Το FDR χρειάζεται έναν entry point — το MAIN process.»

Βήμα 5: (Προαιρετικό) Προσθέτω asserts

«Μπορώ να ζητήσω deadlock freedom, trace refinement, κλπ.»

★ 4. TEACHING Vending Machine

«Ας φτιάξουμε το πιο κλασικό μοντέλο: vending machine.
Θέλω ένα μηχάνημα που:

- περιμένει coin
- μετά μπορεί να δώσει choc ή toffee
- και μετά ξαναγυρνάει στην αρχή.»

Ο κώδικας:

```
channel coin, choc, toffee
```

```
VENDING =  
  coin ->  
    ( choc -> VENDING  
      []  
      toffee -> VENDING  
    )
```

```
MAIN = VENDING
```

Επεξήγηση κάθε γραμμής

```
channel coin, choc, toffee
```

«Δηλώνω τα events μου. Αυτά είναι observable στο FDR.»

```
VENDING = coin -> (...)
```

«Το μηχάνημα ΠΡΩΤΑ περιμένει ένα coin.»

```
(choc -> VENDING [] toffee -> VENDING)
```

«Μετά το coin, αφήνω την **environment** (phase του CSP) να διαλέξει: είτε choc είτε toffee.»

```
MAIN = VENDING
```

«Αυτό θα simulate-άρουμε στο Probe.»

Κάνεις click στο MAIN → **Probe**:

- Πατάς coin
- Μετά το Probe δείχνει δύο πράγματα: choc ή toffee
- Με το που διαλέξεις ένα, το σύστημα γυρνά στην αρχική κατάσταση

«Βλέπετε; Η recursion μας κρατάει το μηχάνημα “ζωντανό”.»

☆ 5. BANK SYSTEM (Example 2)

Σύνδεση θεωρίας + state + guards.

Μοντελοποίηση τραπεζικού λογαριασμού με δύο καταστάσεις:

- διαθέσιμα χρήματα (avail)
- μεγάλα deposits σε hold (held)
Μόνο όταν συμβεί clear_large, τα held γίνονται διαθέσιμα.»

Θα μοντελοποιήσουμε επίσης fraud_withdraw, δηλαδή κλοπή από compromised account.

Code

```

channel deposit_small, deposit_large, withdraw, query, clear_large,
fraud_withdraw
channel show_balance : Nat

ACCOUNT(avail, held) =

  deposit_small ->
    ACCOUNT(avail + 1, held)
  []

  deposit_large ->
    ACCOUNT(avail, held + 1)
  []

  (avail > 0 & withdraw ->
    ACCOUNT(avail - 1, held))
  []

  query ->
    show_balance!avail ->
    ACCOUNT(avail, held)
  []

  (held > 0 & clear_large ->
    ACCOUNT(avail + held, 0))
  []

  (avail > 0 & fraud_withdraw ->
    ACCOUNT(avail - 1, held))

BANK = ACCOUNT(0, 0)
MAIN = BANK

```

1. Τα channels

«Εδώ δηλώνω όλα τα events του λογαριασμού.»

2. Το state του process

«Το ACCOUNT(avail, held) είναι ουσιαστικά ένα FSM (Finite State Machine) όπου το state είναι δύο αριθμοί.»

3. deposit_small

«Αυξάνει άμεσα το διαθέσιμο ποσό.»

4. deposit_large

«Αυτά τα χρήματα πάνε σε hold — δεν μπορείς να τα κάνεις withdraw.»

5. Guarded withdraw

«Το withdraw επιτρέπεται ΜΟΝΟ αν avail > 0.»

6. Query + show_balance

«Το query είναι request, το show_balance είναι response.»

7. Clear_large

«Όταν η τράπεζα εγκρίνει το μεγάλο deposit, τα χρήματα γίνονται διαθέσιμα.»

8. Fraud_withdraw

«Αυτό μοντελοποιεί unauthorized stealing.»

Probe

- Ξεκινάς με `deposit_small` → `availability = 1`
- Κάνεις `query` → εμφανίζεται το event `show_balance.1` στο Probe
- Κάνεις `deposit_large` → `held = 1`
- Τώρα `withdraw` επιτρέπεται ΜΟΝΟ για το `avail`
- Κάνεις `clear_large` → `held` μεταφέρεται στο `avail`
- Δείχνεις πώς ένα `fraud_withdraw` μειώνει το `avail`

Αυτό βοηθά να δουν το state machine “ζωντανό”.

★ 6. Verification με FDR4

Deadlock freedom:

```
assert MAIN : [deadlock free [F]]
```

«Αυτό λέει στο FDR: βρες αν υπάρχει κατάσταση όπου κανένα event δεν μπορεί να γίνει.»

Refinement checking

```
SPEC = RUN({deposit_small, deposit_large, withdraw, query})
assert MAIN [T= SPEC
```

«Εδώ ζητάω: όλα τα traces επιτρέπονται;
Αν όχι, το FDR θα δώσει counterexample.»
